SYSTEM DESIGN

Consistency Models Visual Guide

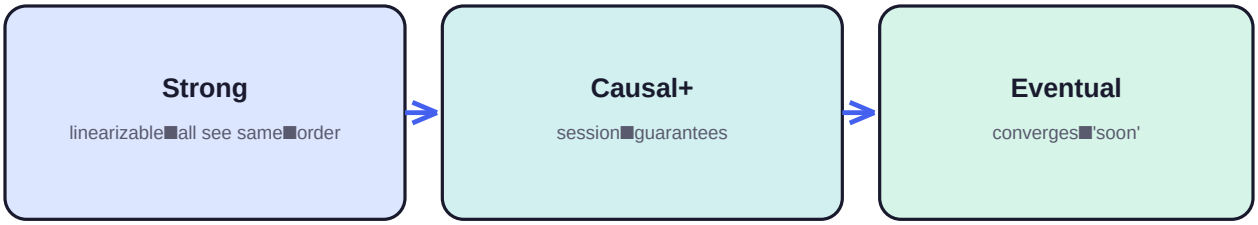
A spectrum from strong to eventual:
guarantees you can actually promise users.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

The Spectrum

Strong consistency is expensive; eventual is cheap. Most systems live between.



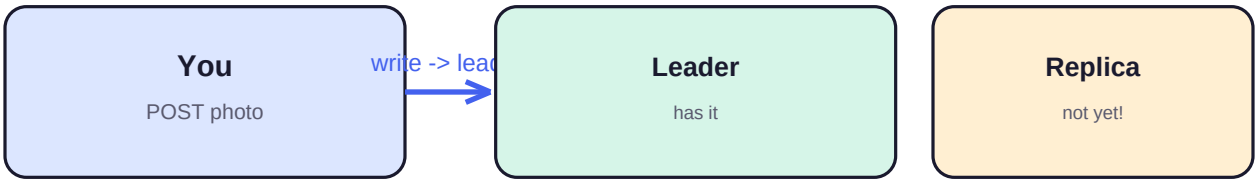
Stronger -> more coordination -> higher latency, lower availability. Weaker -> the reverse.

Eventual consistency only promises CONVERGENCE: if writes stop, all replicas eventually agree. 'Eventually' is doing a lot of work there.

STEP 2/6

Read-Your-Writes

After I write, I must see my own write -- even on a replica.



- The classic violation: post a photo, refresh, it is GONE (read hit a stale replica) -- users call this a bug.
- Fixes: route the writer's reads to the leader for a window, or version-stamp and wait.
- This is a SESSION guarantee: it only constrains what YOU see, not what everyone sees.

STEP 3/6

Monotonic Reads

Once you have seen version 5, you never go back to version 3.

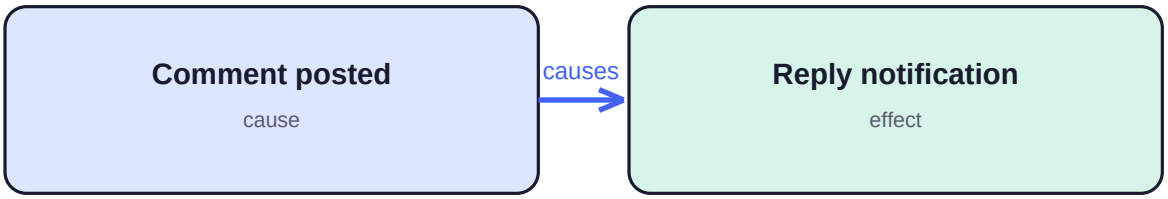
```
read 1 -> v5 (replica A)     read 2 -> v3 (replica B)   <- VIOLATION
read 1 -> v5 (replica A)     read 2 -> v5 (replica A)   <- OK
```

- Time going backwards confuses users AND breaks logic (a deleted item reappears).
- Fix: sticky reads per session (same replica), or track a version watermark per client.
- Cheap to provide, embarrassing to lack -- include it in every session-consistency bundle.

STEP 4/6

Causal Consistency

If A caused B, everyone sees A before B. Unrelated writes may interleave.



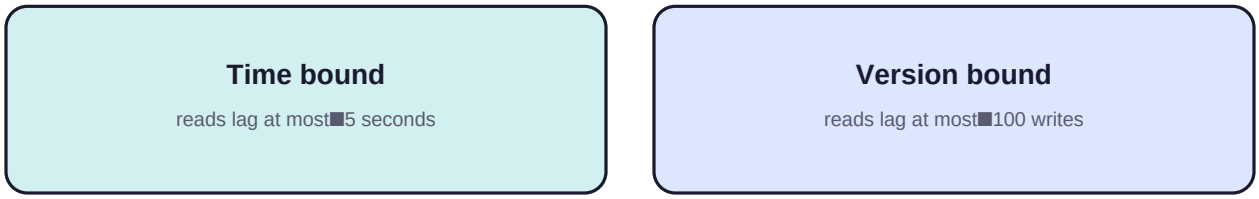
- The violation: you get notified of a reply to a comment you cannot see yet -- causality broken.
- Stronger than eventual, cheaper than linearizable: track causal deps (vector clocks!) not total order.
- The sweet spot for social feeds, chat, and collaboration: conversations stay coherent.

STEP 5/6

Bounded Staleness

Eventual consistency with an SLA: 'no older than X seconds or Y versions'.

Cosmos DB style: pick staleness bound per workload.



- Turns 'eventually' into a number you can put in an SLA and alert on.
- Implementation: replicas reject reads older than the bound (or the writer waits).
- Strong session: read-your-writes + monotonic reads bundled -- the pragmatic default.

STEP 6/6

Choosing Your Model

Match the guarantee to the cost of being wrong.

- Money, inventory, locks: strong consistency -- being wrong costs real dollars.
- Feeds, profiles, catalogs: eventual + read-your-writes -- staleness is invisible here.
- Chat, comments, collaboration: causal -- conversations must make sense.
- Dashboards, analytics: bounded staleness -- '5 minutes old' is a fine SLA.
- Rule: buy the WEAKEST consistency your UX tolerates; spend the savings on availability.

STEP ✓I✓

Key Takeaways

Consistency Models — Remember

- Strong = total order for everyone; eventual = convergence with no timeline -- most systems need the middle.
- Read-your-writes: your own writes are immediately visible to you (route writers to the leader).
- Monotonic reads: never show older data after newer -- time must not go backwards.
- Causal consistency: causes precede effects for everyone -- the social-feed sweet spot.
- Bounded staleness puts an SLA on 'eventually'; buy the weakest model your UX tolerates.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev